

Guía de implementación del frontend

Cómo se construyó la aplicación web de StockFlow siguiendo las guías G01-G10

StockFlow — Inventario, compras internas y movimientos de almacén (PA-06)

React 19 · TypeScript 6 · Vite 8 · React Router 8 · Vitest 5 · Testing Library · WebStorm

Para quién	Integrantes del equipo que deben entender, explicar o volver a construir el frontend
Punto de partida	Las guías G01-G10 de ParkFlow360 (Cliente → Vehículo) y el backend StockFlow ya funcionando
Resultado	webstorm/stockflow-frontend : dashboard y CRUD completo de Categoría 1:N Producto
Para ejecutarlo	Ver la Guía de ejecución , capítulo 4 (esta guía explica cómo se construyó)
Fecha	5 de octubre de 2026 · rama Hector · commits 46150a5 , 5af129a y a076d23

Todo el código que aparece en esta guía se copió automáticamente de los archivos del proyecto. El proyecto compila, pasa 28 pruebas y se probó con un navegador real contra el backend.

Índice

1. Antes de empezar	3
2. Preparación	5
Preparación 1 · Node.js, npm y WebStorm	5
Preparación 2 · El backend debe responder y permitir CORS	5
3. Construir el frontend, paso a paso	7
Paso 1 · Crear el proyecto con Vite (G01)	7
Paso 2 · Limpiar la plantilla y crear las carpetas (G01, G02)	8
Paso 3 · Configuración: la URL del backend (G05, G10)	11
Paso 4 · Modelos y tipos: el contrato con el backend (G03, G04)	12
Paso 5 · El cliente HTTP: apiClient (G05)	14
Paso 6 · Services: un objeto por entidad (G05, G06)	16
Paso 7 · Un hook para cargar listas (G08)	17
Paso 8 · Layout, menú y rutas (G02)	19
Paso 9 · Estilos: design tokens y CSS global (G02, G03, G09)	22
Paso 10 · Componentes de UI reutilizables (G09)	23
Paso 11 · La tabla (G03)	26
Paso 12 · Formulario controlado y validación (G04)	27
Paso 13 · La Page: CRUD completo de Categoría (G06)	30
Paso 14 · La entidad hija: Producto y la relación 1:N (G07)	33
Paso 15 · Búsqueda, filtros y paginación (G09)	35
Paso 16 · Dashboard y errores globales (G10)	37
Paso 17 · Pruebas automáticas con Vitest (G10)	39
Paso 18 · Verificación final contra el backend real	41
4. Diferencias con las guías y por qué	42
5. Cómo explicarlo en la defensa	43
6. Problemas frecuentes al implementar	45
7. Glosario	46

1. Antes de empezar

Las guías G01 a G10 del curso construyen el frontend de **ParkFlow360**, un sistema de estacionamientos con la relación **Cliente 1:N Vehículo**. StockFlow tiene la misma forma: **Categoría 1:N Producto**. Esta guía muestra cómo se aplicó cada guía a StockFlow, archivo por archivo, y explica las decisiones que se tomaron cuando una guía tenía un error o una versión desactualizada.

No hace falta haber leído las guías para seguirla, pero conviene tenerlas a mano: cada paso dice de qué guía sale (por ejemplo **G05**).

Si sólo quieres **ejecutar** el frontend, no necesitas esta guía: usa el capítulo 4 de la **Guía de ejecución**. Esta guía es para **entender** el código y poder explicarlo en la defensa.

Cómo leer cada paso

```
src/ruta/del/archivo.ts
```

```
// Los bloques oscuros con un nombre arriba son el código REAL del proyecto,  
// copiado automáticamente de webstorm/stockflow-frontend al generar esta guía.  
// Una línea muy larga continúa en la siguiente, marcada con ↵
```

Los bloques oscuros sin nombre son comandos para la terminal.

✓ Qué deberías ver

Cómo comprobar que el paso quedó bien.

⚠ Si falla

Los errores típicos de ese paso y su solución.

i Para entenderlo

El concepto detrás del código (lo que te pueden preguntar en la defensa).

De ParkFlow360 a StockFlow

Todo lo que en las guías se llama Cliente o Vehículo, en StockFlow se llama así:

ParkFlow360 (guías)	StockFlow	Por qué
Cliente (padre)	Categoría	Un cliente tiene muchos vehículos; una categoría tiene muchos productos.
Vehículo (hijo)	Producto	El hijo guarda la referencia al padre (clave foránea).
clienteId	categoriaId	Así viaja la relación en el JSON: el producto no contiene el objeto categoría.
placa	codigo	El identificador de negocio único (UNIQUE en PostgreSQL), distinto del id .
/api/clientes, /api/vehiculos	/api/categorias, /api/productos	Endpoints REST del backend Spring Boot.
Dashboard de ocupación	Dashboard de inventario	Métricas de categorías y productos y la lista de stock bajo el mínimo (RF-20).

ParkFlow360 (guías)	StockFlow	Por qué
VITE_API_BASE_URL (G01)	VITE_API_URL	G05 y G10 usan VITE_API_URL; se unificó en ese nombre.

Qué aporta cada guía

Guía	Tema	Qué se hizo en StockFlow (archivos principales)
G01	Proyecto Vite en WebStorm	Proyecto <code>stockflow-frontend</code> con la plantilla <code>react-ts</code> ; estructura de carpetas por <i>feature</i> (pasos 1 y 2).
G02	Navegación y layout	<code>main.tsx</code> , <code>AppRouter.tsx</code> , <code>MainLayout.tsx</code> , <code>Sidebar.tsx</code> , <code>Header.tsx</code> (paso 8).
G03	Modelos, datos simulados y tablas	<code>Categoria.ts</code> , <code>Producto.ts</code> , <code>*.mock.ts</code> , <code>CategoriaTable.tsx</code> , <code>ProductoTable.tsx</code> (pasos 4 y 11).
G04	Formularios y validación	<code>CategoriaForm.tsx</code> , <code>ProductoForm.tsx</code> , <code>categoriaValidation.ts</code> , <code>productoValidation.ts</code> (paso 12).
G05	Conexión con Spring Boot	<code>.env.development</code> , <code>env.ts</code> , <code>apiClient.ts</code> , <code>services</code> (pasos 3, 5 y 6).
G06	CRUD completo del padre	<code>CategoriasPage.tsx</code> : GET, GET por id, POST, PUT y DELETE con confirmación (paso 13).
G07	CRUD del hijo y relación 1:N	<code>ProductosPage.tsx</code> y <code>ProductoForm.tsx</code> : select de categoría y filtro por categoría (paso 14).
G08	Efectos, asincronía y hooks	<code>useAsyncList.ts</code> con <code>AbortController</code> ; <code>useCategorias</code> , <code>useProductos</code> (paso 7).
G09	UI reutilizable, búsqueda, filtros y paginación	<code>components/ui/*</code> , <code>paginar.ts</code> , <code>tokens.css</code> (pasos 9, 10 y 15).
G10	Dashboard, pruebas y build	<code>DashboardPage.tsx</code> , <code>AppErrorBoundary.tsx</code> , Vitest y 28 pruebas (pasos 16 y 17).

2. Preparación

Antes de escribir código, hay que comprobar dos cosas: las herramientas y el backend.

Preparación 1 · Node.js, npm y WebStorm

Vite 8 necesita Node.js 20.19 o superior. En la terminal:

```
node -v
npm -v
```

✓ Qué deberías ver

En la computadora del proyecto: **v26.10.0** y **12.2.0**. WebStorm se abre desde JetBrains Toolbox.

⚠ Si falla

Si **node** no existe: `sudo pacman -S nodejs npm`.

Preparación 2 · El backend debe responder y permitir CORS

El frontend no tiene datos propios: todo lo pide al backend. Enciende la base y el backend (Guía de ejecución, capítulo 3) y comprueba:

```
curl -s http://localhost:8080/api/productos
```

El backend ya tenía **CorsConfig** (lo pide G05): sólo permite el origen del frontend y los métodos del CRUD.

`intellij/stockflow-backend/.../shared/web/CorsConfig.java` (fragmento)

```
.allowedOrigins(origenesPermitidos)
.allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
.allowedHeaders("Content-Type", "Accept")
.exposedHeaders("Location") // el frontend puede leer la URL del recurso creado
↪ (201)
```

Lo que **sí faltaba** era el listado general de productos. La G07 lista todos los vehículos con **GET /api/vehiculos**, pero StockFlow sólo listaba productos por categoría. Se agregó el endpoint en las cuatro capas del hexágono (caso de uso, servicio, puerto y adaptador) con sus pruebas, en el commit **46150a5**:

`intellij/stockflow-backend/.../ProductoController.java` (fragmento)

```
/** Listado completo del catálogo, ordenado por código. 200 (lista vacía si no hay productos).
↪ */
@GetMapping
public List<ProductoResponse> listar() {
    return consultar.listarTodos().stream()
        .map(ProductoWebMapper::toResponse)
        .toList();
}
```

✓ Qué deberías ver

Un arreglo JSON con los 5 productos de la semilla, ordenados por código:
`[{"id":4,"categoriaId":3,"codigo":"PRD-EPP-001", ...}, ...].`

⚠ Si falla

Connection refused: el backend está apagado. Un **405 Method Not Allowed** significa que tu backend no tiene el commit **46150a5**: haz **git pull**.

i Para entenderlo

Sin **CorsConfig**, el navegador bloquea las respuestas aunque el backend funcione: Postman y **curl** no aplican CORS, el navegador sí. Por eso una API puede "andar en Postman" y fallar en React.

3. Construir el frontend, paso a paso

Los pasos siguen el orden en que se construyó el proyecto, que es el mismo orden de las guías: primero la estructura, después la conexión con el backend y, al final, la experiencia de uso y las pruebas.

Donde Producto repite exactamente el patrón de Categoría (service, hook, tabla), se muestra sólo Categoría y se indica el archivo equivalente.

Paso 1 · Crear el proyecto con Vite (G01)

En la carpeta `webstorm` del repositorio:

```
cd ~/Desktop/StockFlow/webstorm
npm create vite@latest stockflow-frontend -- --template react-ts
cd stockflow-frontend
npm install
```

Después se agregan el router (G02) y las herramientas de pruebas (G10):

Terminal

```
npm install react-router
npm install -D vitest jsdom @testing-library/react @testing-library/dom \
  @testing-library/jest-dom @testing-library/user-event
```

Así queda `package.json` (los dos scripts de pruebas se agregaron a mano en el paso 17):

package.json

```
{
  "name": "stockflow-frontend",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "tsc -b && vite build",
    "lint": "oxlint",
    "preview": "vite preview",
    "test": "vitest",
    "test:run": "vitest run"
  },
  "dependencies": {
    "react": "^19.2.8",
    "react-dom": "^19.2.8",
    "react-router": "^8.4.0"
  },
  "devDependencies": {
    "@testing-library/dom": "^10.4.2",
    "@testing-library/jest-dom": "^7.0.1",
    "@testing-library/react": "^16.3.3",
    "@testing-library/user-event": "^14.6.7",
    "@types/node": "^24.13.3",
    "@types/react": "^19.2.18",
    "@types/react-dom": "^19.2.7",
    "@vitejs/plugin-react": "^6.1.1",
    "jsdom": "^30.1.2",
    "oxlint": "^1.81.0",
    "typescript": "~6.0.2",
    "vite": "^8.3.0",
    "vitest": "^5.0.3"
  }
}
```

✓ Qué deberías ver

`npm run dev` abre la página de bienvenida de Vite en <http://localhost:5173>.

⚠ Si falla

Si `npm create` hace preguntas, elige **React** y luego **TypeScript**. Si no aparece `"lint": "oxlint"`, tu Vite es más antiguo: no es un problema, sólo cambia el linter.

i Para entenderlo

Vite cumple dos funciones: en desarrollo sirve los archivos y recarga la página al guardar; para producción (`npm run build`) junta todo en unos pocos archivos optimizados dentro de `dist/`. Se instala `react-router` y **no** `react-router-dom`: desde la versión 7 son el mismo paquete, y G02 usa el nombre nuevo.

Paso 2 · Limpiar la plantilla y crear las carpetas (G01, G02)

Se borra el ejemplo de Vite y se crea la estructura por *feature* (funcionalidad):

```
rm -r src/App.tsx src/App.css src/index.css src/assets
mkdir -p src/{api,app,config,layouts,pages,routes,styles,test} \
  src/components/{common,ui} src/shared/{hooks,utils} \
  src/features/inventario/{models,services}
cd src/features
mkdir -p {categorias,productos}/{models,types,services,hooks} \
  {categorias,productos}/{utils,components,pages,data}
cd ../../
```

Estructura final de src/ (sin los archivos de prueba)

```
src
├── api
│   └── apiClient.ts
├── app
│   ├── AppErrorBoundary.tsx
│   └── App.tsx
├── components
│   ├── common
│   │   ├── Header.tsx
│   │   └── Sidebar.tsx
│   └── ui
│       ├── Alert.tsx
│       ├── Button.tsx
│       ├── ConfirmDialog.tsx
│       ├── EmptyState.tsx
│       ├── MetricCard.tsx
│       ├── Modal.tsx
│       ├── Pagination.tsx
│       ├── SearchInput.tsx
│       ├── SelectFilter.tsx
│       └── StatusBadge.tsx
├── config
│   └── env.ts
├── features
│   ├── categorias
│   │   ├── components
│   │   │   ├── CategoriaForm.tsx
│   │   │   └── CategoriaTable.tsx
│   │   ├── data
│   │   │   └── categorias.mock.ts
│   │   ├── hooks
│   │   │   └── useCategorias.ts
│   │   ├── models
│   │   │   └── Categoria.ts
│   │   ├── pages
│   │   │   └── CategoriasPage.tsx
│   │   ├── services
│   │   │   └── categoriaService.ts
│   │   ├── types
│   │   │   ├── CategoriaFormData.ts
│   │   │   └── CategoriaRequests.ts
│   │   └── utils
│   │       └── categoriaValidation.ts
│   └── inventario
│       ├── models
│       │   └── Existencia.ts
│       └── services
│           └── inventarioService.ts
```

Estructura final de src/ (sin los archivos de prueba) (continuación)

```

├── productos
│   ├── components
│   │   ├── ProductoForm.tsx
│   │   └── ProductoTable.tsx
│   ├── data
│   │   └── productos.mock.ts
│   ├── hooks
│   │   └── useProductos.ts
│   ├── models
│   │   └── Producto.ts
│   ├── pages
│   │   └── ProductosPage.tsx
│   ├── services
│   │   └── productoService.ts
│   ├── types
│   │   ├── ProductoFormData.ts
│   │   └── ProductoRequests.ts
│   └── utils
│       └── productoValidation.ts
├── layouts
│   └── MainLayout.tsx
├── main.tsx
├── pages
│   ├── DashboardPage.tsx
│   └── NotFoundPage.tsx
├── routes
│   └── AppRouter.tsx
├── shared
│   ├── hooks
│   │   └── useAsyncList.ts
│   └── utils
│       ├── isAbortError.ts
│       └── paginar.ts
├── styles
│   ├── global.css
│   └── tokens.css
├── test
│   └── setup.ts
└── vite-env.d.ts

```

✓ Qué deberías ver

En WebStorm, `src/` con las carpetas de arriba. Los archivos se crean en los pasos siguientes.

i Para entenderlo

Cada carpeta de `features/` tiene todo lo de **una** entidad: su modelo, cómo se habla con el backend (`services`), cómo se carga (`hooks`), cómo se valida (`utils`) y cómo se ve (`components`, `pages`). Lo que usan varias entidades va en `shared/` (lógica) o `components/ui/` (piezas visuales). Es el mismo criterio que el backend hexagonal: si mañana se agrega **Ubicación**, se crea `features/ubicaciones/` sin tocar las demás.

Paso 3 · Configuración: la URL del backend (G05, G10)

La dirección del backend no se escribe en el código: se lee de una variable de entorno.

`.env.development`

```
# URL de la API en desarrollo (npm run dev). Termina en /api: los services agregan /categorias,
  ↳ /productos...
# Las variables VITE_ quedan visibles en el navegador: NUNCA pongas contraseñas ni tokens aquí.
VITE_API_URL=http://localhost:8080/api
```

`.env.production`

```
# URL de la API al compilar para producción (npm run build). Cámbiala por el dominio real del
  ↳ backend.
VITE_API_URL=https://api.stockflow.example.com/api
```

TypeScript necesita saber que esa variable existe y que es un texto:

`src/vite-env.d.ts`

```
/// <reference types="vite/client" />

// G05: tipado de las variables de entorno que usa StockFlow
interface ImportMetaEnv {
  readonly VITE_API_URL: string;
}

interface ImportMeta {
  readonly env: ImportMetaEnv;
}
```

Y un solo archivo la lee. Si falta, la aplicación avisa al arrancar, no en cada petición:

`src/config/env.ts`

```
// G05/G10: un solo lugar lee la configuración. Si falta, el error aparece al arrancar y no en cada
  ↳ petición.
export const API_URL: string = import.meta.env.VITE_API_URL;

if (!API_URL) {
  throw new Error('Falta VITE_API_URL: crea .env.development (ver .env.example) y reinicia npm run
    ↳ dev');
}
```

✓ Qué deberías ver

`npm run dev` arranca sin el error *Falta VITE_API_URL*.

⚠ Si falla

Si cambias un `.env` y no pasa nada: Vite los lee **sólo al arrancar**. `Ctrl+C` y `npm run dev` de nuevo.

i Para entenderlo

Vite elige el archivo según el modo: `npm run dev` usa `.env.development` y `npm run build` usa `.env.production`. Sólo las variables con prefijo `VITE_` llegan al navegador, y llegan **visibles** para cualquiera (F12): nunca se pone una contraseña ahí.

Paso 4 · Modelos y tipos: el contrato con el backend (G03, G04)

Se separan tres tipos de datos: lo que **responde** el backend (modelo), lo que se le **envía** (request) y lo que el usuario está **escribiendo** (form data).

src/features/categorias/models/Categoria.ts

```
/**
 * Contrato de CategoriaResponse del backend (GET /api/categorias).
 * No es la entidad JPA: es lo que la API decide exponer (sin createdAt ni updatedAt).
 */
export interface Categoria {
  id: number;
  codigo: string;
  nombre: string;
  descripcion: string | null;
  activo: boolean;
}
```

src/features/productos/models/Producto.ts

```
/** Coincide con el enum UnidadMedida de Java y con el CHECK ck_producto_unidad_medida de
  ↳ PostgreSQL. */
export const UNIDADES_MEDIDA = ['UNIDAD', 'CAJA', 'PAQUETE', 'KILOGRAMO', 'LITRO', 'METRO'] as
  ↳ const;
export type UnidadMedida = (typeof UNIDADES_MEDIDA)[number];

/**
 * Contrato de ProductoResponse del backend.
 * La relación con Categoria (1:N) viaja como categoriaId: el producto NO contiene el objeto
  ↳ categoría.
 */
export interface Producto {
  id: number;
  categoriaId: number;
  codigo: string;
  nombre: string;
  descripcion: string | null;
  unidadMedida: UnidadMedida;
  stockMinimoDefault: number;
  precioReferencial: number | null;
  activo: boolean;
}
```

src/features/categorias/types/CategoriaRequests.ts

```
/** CrearCategoriaRequest del backend: no lleva id (lo genera PostgreSQL) ni activo (nace activa).
  ↳ */
export interface CategoriaCreateRequest {
  codigo: string;
  nombre: string;
  descripcion: string | null;
}

/** ActualizarCategoriaRequest: reemplazo completo (PUT). El id viaja en la URL; activo es
  ↳ obligatorio. */
export interface CategoriaUpdateRequest extends CategoriaCreateRequest {
  activo: boolean;
}
```

src/features/productos/types/ProductoFormData.ts

```
import type { UnidadMedida } from '../models/Producto';

/**
 * G04/G07: estado del formulario. categoriaId, stockMinimoDefault y precioReferencial son string
 * porque los <input>/<select> entregan texto; se validan y recién después se convierten a number.
 */
export interface ProductoFormData {
  categoriaId: string;
  codigo: string;
  nombre: string;
  descripcion: string;
  unidadMedida: UnidadMedida;
  stockMinimoDefault: string;
  precioReferencial: string;
  activo: boolean;
}
```

✓ Qué deberías ver

Los nombres coinciden **exactamente** con el JSON del backend: compara con `curl http://localhost:8080/api/productos/1`.

⚠ Si falla

Si un campo se llama distinto (por ejemplo `categoria_id`), TypeScript no avisa: el valor llega `undefined` en tiempo de ejecución. Copia los nombres del JSON real.

i Para entenderlo

La interfaz `Producto` es el `ProductoResponse` de Java, **no** la entidad JPA: no trae `createdAt` porque la API no lo expone.

En el formulario, `stockMinimoDefault` es `string` porque un `<input>` siempre entrega texto: se valida como texto y se convierte a número **recién al enviar**. `UNIDADES_MEDIDA ... as const` permite usar la lista en el `<select>` y, a la vez, derivar el tipo `UnidadMedida`.

Paso 5 · El cliente HTTP: apiClient (G05)

Todas las peticiones pasan por una sola función, `apiFetch`. Es el archivo más importante para la defensa:

`src/api/apiClient.ts`

```
import { API_URL } from '../config/env';

/**
 * Formato de error del backend StockFlow (GlobalExceptionHandler -> ApiErrorResponse).
 * Todas las respuestas 400/404/409/422/500 traen esta forma.
 */
export interface ApiErrorResponse {
  timestamp: string;
  status: number;
  error: string;
  message: string;
  path: string;
  fieldErrors: Record<string, string>;
}

/**
 * Error HTTP con el status y los errores por campo que envió el backend.
 * Nota: los campos se declaran explícitamente porque el tsconfig de Vite usa erasableSyntaxOnly,
 * que no permite "parameter properties" como constructor(public status: number).
 */
export class ApiError extends Error {
  readonly status: number;
  readonly fieldErrors: Record<string, string>;

  constructor(status: number, message: string, fieldErrors: Record<string, string> = {}) {
    super(message);
    this.name = 'ApiError';
    this.status = status;
    this.fieldErrors = fieldErrors;
  }
}

/** Lee el cuerpo de error: si es el JSON de StockFlow usa su message; si no, el texto o el
  ↳ statusText. */
async function construirError(response: Response): Promise<ApiError> {
  const texto = await response.text();
  try {
    const cuerpo = JSON.parse(texto) as Partial<ApiErrorResponse>;
    if (cuerpo && typeof cuerpo.message === 'string') {
      return new ApiError(response.status, cuerpo.message, cuerpo.fieldErrors ?? {});
    }
  } catch {
    // No era JSON (por ejemplo, un error de un proxy): se usa el texto tal cual
  }
  return new ApiError(response.status, texto || response.statusText || `Error HTTP
  ↳ ${response.status}`);
}
```

src/api/apiClient.ts (continuación)

```

/**
 * Única puerta HTTP del frontend (G05): URL base, cabecera JSON, verificación de response.ok,
 * conversión de errores y lectura del cuerpo. fetch NO rechaza la Promise ante un 404 o un 500:
 * por eso se revisa response.ok aquí, en un solo lugar.
 */
export async function apiFetch<T>(endpoint: string, options: RequestInit = {}): Promise<T> {
  const headers = new Headers(options.headers);
  if (options.body && !headers.has('Content-Type')) {
    headers.set('Content-Type', 'application/json');
  }
  headers.set('Accept', 'application/json');

  const response = await fetch(`${API_URL}${endpoint}`, { ...options, headers });

  if (!response.ok) {
    throw await construirError(response);
  }
  if (response.status === 204) {
    return undefined as T; // DELETE exitoso: no hay cuerpo que leer
  }
  return (await response.json()) as T;
}

/** Mensaje legible para mostrar en pantalla a partir de cualquier error. */
export function mensajeDeError(error: unknown, porDefecto = 'Ocurrió un error inesperado'): string {
  if (error instanceof ApiError) return error.message;
  if (error instanceof TypeError) {
    // fetch lanza TypeError cuando no llega a ningún servidor (backend apagado, CORS, URL mal
    ↪ escrita)
    return 'No se pudo conectar con el backend. ¿Está corriendo StockFlowApplication en el puerto
    ↪ 8080?';
  }
  return error instanceof Error ? error.message : porDefecto;
}

```

✓ Qué deberías ver

Todavía nada visible: lo usan los services del paso siguiente. Su prueba (`apiClient.test.ts`, paso 17) comprueba los casos 200, 204 y 400.

⚠ Si falla

Si copias la `ApiError` de la G05 (`constructor(public status: number, ...)`), TypeScript responde `TS1294: This syntax is not allowed when 'erasableSyntaxOnly' is enabled`. Declara los campos como arriba.

i Para entenderlo

fetch no lanza error ante un 404 o un 500: la promesa se cumple igual y hay que mirar `response.ok`. Si cada pantalla lo hiciera por su cuenta, alguna lo olvidaría. Aquí se hace una sola vez.

El cuerpo de error es el `ApiErrorResponse` del `GlobalExceptionHandler` del backend; `fieldErrors` trae los errores por campo de un 400, y el formulario los muestra junto a cada input.

Si el backend está apagado, `fetch` sí lanza un `TypeError (Failed to fetch)`; `mensajeDeError` lo traduce a un mensaje entendible.

Paso 6 · Services: un objeto por entidad (G05, G06)

Los services conocen las rutas; los componentes sólo dicen **qué** quieren hacer.

`src/features/categorias/services/categoriaService.ts`

```
import { apiFetch } from '../../../api/apiClient';
import type { Categoria } from '../../../models/Categoria';
import type { CategoriaCreateRequest, CategoriaUpdateRequest } from '../../../types/CategoriaRequests';

/**
 * G06: el único lugar que conoce las rutas de categorías. Los componentes expresan intenciones
 * (listar, actualizar...) y no construyen URLs ni headers.
 */
export const categoriaService = {
  listar(signal?: AbortSignal): Promise<Categoria[]> {
    return apiFetch<Categoria[]>('/categorias', { signal });
  },
  obtenerPorId(id: number, signal?: AbortSignal): Promise<Categoria> {
    return apiFetch<Categoria>(`/categorias/${id}`, { signal });
  },
  crear(data: CategoriaCreateRequest): Promise<Categoria> {
    return apiFetch<Categoria>('/categorias', { method: 'POST', body: JSON.stringify(data) });
  },
  actualizar(id: number, data: CategoriaUpdateRequest): Promise<Categoria> {
    return apiFetch<Categoria>(`/categorias/${id}`, { method: 'PUT', body: JSON.stringify(data) });
  },
  /** 204 si se eliminó; 409 si la categoría tiene productos (FK sin CASCADE en PostgreSQL). */
  eliminar(id: number): Promise<void> {
    return apiFetch<void>(`/categorias/${id}`, { method: 'DELETE' });
  },
};
```

`productoService.ts` es idéntico con `/productos`, e `inventarioService.ts` tiene una sola consulta para el dashboard:

`src/features/inventario/services/inventarioService.ts`

```
import { apiFetch } from '../../../api/apiClient';
import type { Existencia } from '../../../models/Existencia';

export const inventarioService = {
  /** RF-20: existencias en o bajo su stock mínimo, por producto y ubicación. */
  stockBajoMinimo(signal?: AbortSignal): Promise<Existencia[]> {
    return apiFetch<Existencia[]>('/inventario/stock?soloBajoMinimo=true', { signal });
  },
};
```

✓ Qué deberías ver

Ningún componente contiene `fetch` ni la palabra `http`: búscalo con `Ctrl+Shift+F` en WebStorm.

i Para entenderlo

Los métodos de lectura reciben un `AbortSignal` opcional: así, si el usuario sale de la pantalla antes de que llegue la respuesta, la petición se cancela (paso 7).

Paso 7 · Un hook para cargar listas (G08)

Cargar datos siempre necesita lo mismo: el dato, si está cargando y si hubo error. Se escribió una vez como hook genérico:

src/shared/hooks/useAsyncList.ts

```
import { useCallback, useEffect, useState } from 'react';
import { mensajeDeError } from '../../../api/apiClient';
import { isAbortError } from '../../../utils/isAbortError';

type Loader<T> = (signal: AbortSignal) => Promise<T[]>;

/**
 * G08: patrón genérico para cargar una colección del backend con estado data/loading/error.
 * - El Effect sincroniza la UI con un sistema externo (la API) y devuelve un cleanup que aborta.
 * - loader debe ser una referencia estable (definida fuera del componente), si no el Effect se
 *   ↪ repetiría.
 * - reload() vuelve a ejecutar la carga cambiando reloadToken.
 */
export function useAsyncList<T>(loader: Loader<T>) {
  const [data, setData] = useState<T[]>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState('');
  const [reloadToken, setReloadToken] = useState(0);

  const reload = useCallback(() => setReloadToken((v) => v + 1), []);

  useEffect(() => {
    const controller = new AbortController();
    const load = async () => {
      try {
        setLoading(true);
        setError('');
        setData(await loader(controller.signal));
      } catch (err) {
        if (!isAbortError(err)) setError(mensajeDeError(err, 'Error de carga'));
      } finally {
        if (!controller.signal.aborted) setLoading(false);
      }
    };
    void load();
    return () => controller.abort();
  }, [loader, reloadToken]);

  return { data, setData, loading, error, setError, reload };
}
```

Y cada entidad lo usa en una línea:

src/features/categorias/hooks/useCategorias.ts

```
import { useAsyncList } from '../../../shared/hooks/useAsyncList';
import type { Categoria } from '../../../models/Categoria';
import { categoriaService } from '../../../services/categoriaService';

/** G08: hook de dominio. useAsyncList conoce el patrón; categoriaService conoce la ruta HTTP. */
export function useCategorias() {
  return useAsyncList<Categoria>(categoriaService.listar);
}
```

src/shared/utils/isAbortError.ts

```
/** G08: abortar una petición (al desmontar o cambiar de pantalla) no es un error del sistema. */
export function isAbortError(error: unknown): boolean {
  return error instanceof DOMException && error.name === 'AbortError';
}
```

✓ Qué deberías ver

En el navegador, **F12** → **Network**: al entrar a Categorías aparece **una** petición **categorias** con estado 200 (en desarrollo puede verse una primera *cancelada*: es StrictMode, ver abajo).

⚠ Si falla

Si la petición se repite sin parar, el **loader** cambia en cada render: pasa una función definida **fuera** del componente (como **categoriaService.listar**), nunca una flecha creada dentro.

i Para entenderlo

useEffect sirve para **sincronizar** el componente con algo externo (aquí, la API). Su función de retorno (*cleanup*) aborta la petición si el componente se desmonta.

En desarrollo, **StrictMode** monta, desmonta y vuelve a montar cada componente para detectar Effects sin cleanup: por eso se ve una petición cancelada y otra exitosa. En producción hay una sola.

Paso 8 · Layout, menú y rutas (G02)

La aplicación es una **SPA** (*single page application*): hay un solo HTML y React cambia el contenido según la URL.

src/main.tsx

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import { BrowserRouter } from 'react-router';
import App from './app/App';
import { AppErrorBoundary } from './app/AppErrorBoundary';
import './styles/tokens.css';
import './styles/global.css';

// G01/G02/G10: punto de entrada. React se monta en el <div id="root"> de index.html.
// StrictMode (sólo en desarrollo) ejecuta los Effects dos veces para detectar Effects sin cleanup.
createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <AppErrorBoundary>
      <BrowserRouter>
        <App />
      </BrowserRouter>
    </AppErrorBoundary>
  </StrictMode>,
);
```

src/routes/AppRouter.tsx

```
import { Route, Routes } from 'react-router';
import CategoriasPage from '../features/categorias/pages/CategoriasPage';
import ProductosPage from '../features/productos/pages/ProductosPage';
import MainLayout from '../layouts/MainLayout';
import DashboardPage from '../pages/DashboardPage';
import NotFoundPage from '../pages/NotFoundPage';

/**
 * G02: qué página corresponde a cada URL. Las rutas hijas no empiezan con "/" porque cuelgan
 * de la ruta de layout. Ojo: /categorias es una ruta del NAVEGADOR, no el endpoint /api/categorias.
 */
export default function AppRouter() {
  return (
    <Routes>
      <Route element={<MainLayout />}>
        <Route index element={<DashboardPage />} />
        <Route path="categorias" element={<CategoriasPage />} />
        <Route path="productos" element={<ProductosPage />} />
        <Route path="*" element={<NotFoundPage />} />
      </Route>
    </Routes>
  );
}
```

src/layouts/MainLayout.tsx

```
import { Outlet } from 'react-router';
import Header from '../components/common/Header';
import Sidebar from '../components/common/Sidebar';

/** G02: estructura permanente. Outlet es el hueco donde React Router pinta la página de la URL
    ↪ actual. */
export default function MainLayout() {
  return (
    <div className="app-shell">
      <Sidebar />
      <div className="app-shell__main">
        <Header />
        <main className="app-content">
          <Outlet />
        </main>
      </div>
    </div>
  );
}
```

src/components/common/Sidebar.tsx

```
import { NavLink } from 'react-router';

const menuItems = [
  { to: '/', label: 'Inicio', end: true },
  { to: '/categorias', label: 'Categorías' },
  { to: '/productos', label: 'Productos' },
];

/**
 * G02: menú lateral. NavLink cambia la URL sin recargar la página (SPA) y entrega isActive
 * para marcar la opción actual. end en "/" evita que Inicio quede activo en todas las rutas.
 */
export default function Sidebar() {
  return (
    <aside className="sidebar">
      <div className="sidebar__brand">
        <span className="sidebar__brand-mark">S</span>
        <div>
          <strong>StockFlow</strong>
          <small>Inventario</small>
        </div>
      </div>
      <nav className="sidebar__nav" aria-label="Navegación principal">
        {menuItems.map((item) => (
          <NavLink
            key={item.to}
            to={item.to}
            end={item.end}
            className={({ isActive }) => `sidebar__link${isActive ? ' sidebar__link--active' : ''}`}
            >
              {item.label}
            </NavLink>
          ))}
      </nav>
    </aside>
  );
}
```

✓ Qué deberías ver

Al hacer clic en el menú, la URL cambia a **/categorias** o **/productos** **sin recargar la página** (el ícono de recarga del navegador no gira).

⚠ Si falla

TS2307: Cannot find module 'react-router-dom': copiaste un ejemplo antiguo. Importa desde 'react-router'.

i Para entenderlo

`Outlet` es el hueco del layout donde se pinta la página de la ruta actual; así el menú y el encabezado no se vuelven a crear al navegar. `/categorias` es una ruta **del navegador**; `/api/categorias` es un endpoint **del backend**: son cosas distintas.

Paso 9 · Estilos: design tokens y CSS global (G02, G03, G09)

Los colores, radios y sombras se definen una sola vez como variables CSS (*design tokens*):

`src/styles/tokens.css`

```
/* G09: design tokens. Si cambia la identidad visual, se cambia aquí y no en cada componente. */
:root {
  --color-primary: #17365d;
  --color-primary-strong: #0f2742;
  --color-accent: #2f75b5;
  --color-danger: #b42318;
  --color-danger-soft: #fdecec;
  --color-success: #067647;
  --color-success-soft: #ecfdf3;
  --color-warning: #b54708;
  --color-warning-soft: #fffaeb;
  --color-border: #d0d5dd;
  --color-border-soft: #eaecef;
  --color-surface: #ffffff;
  --color-background: #f3f6fa;
  --color-text: #1d2733;
  --color-muted: #667788;
  --radius-sm: 8px;
  --radius-md: 12px;
  --radius-lg: 16px;
  --shadow-sm: 0 6px 18px rgba(16, 24, 40, 0.05);
  --shadow-md: 0 12px 30px rgba(16, 24, 40, 0.14);
  --space-1: 0.25rem;
  --space-2: 0.5rem;
  --space-3: 0.75rem;
  --space-4: 1rem;
  --space-5: 1.25rem;
  --space-6: 1.5rem;
}
```

`global.css` reúne el CSS de layout (G02), tablas (G03), formularios (G04) y componentes (G09). Un detalle que **no** está en las guías:

`src/styles/global.css (fragmento)`

```
.code-cell { font-family: ui-monospace, 'JetBrains Mono', monospace; font-weight: 700; color:
  ↪ #1d4ed8; white-space: nowrap; }
.muted-cell { color: var(--color-muted); }
.danger-cell { color: var(--color-danger); font-weight: 700; }
/* Sin display:flex en el <td>: rompería la celda de tabla. Se separan los botones con margen. */
.actions-cell { white-space: nowrap; }
.actions-cell .btn + .btn { margin-left: 8px; }
```

✓ Qué deberías ver

Los códigos (`PRD-OFI-001`) en una sola línea y los botones **Editar** y **Eliminar** uno al lado del otro.

⚠ Si falla

Si los botones se apilan y el borde de la fila se corta, el `<td>` tiene `display: flex`: eso lo deja de tratar como celda de tabla.

i Para entenderlo

Con tokens, cambiar la identidad visual es editar un archivo. Los componentes usan clases (`btn--danger`), nunca colores sueltos.

Paso 10 · Componentes de UI reutilizables (G09)

Piezas que no saben nada de categorías ni productos: reciben datos por *props* y avisan con *callbacks*.

src/components/ui/Button.tsx

```
import type { ButtonHTMLAttributes } from 'react';

type ButtonVariant = 'primary' | 'secondary' | 'danger' | 'ghost';

type ButtonProps = ButtonHTMLAttributes<HTMLButtonElement> & {
  variant?: ButtonVariant;
  /** Mientras es true, el botón se deshabilita y muestra loadingText: evita doble envío. */
  loading?: boolean;
  loadingText?: string;
};

/** G09: botón genérico. No conoce categorías ni productos; la acción llega por onClick. */
export function Button({
  variant = 'primary',
  loading = false,
  loadingText = 'Procesando...',
  disabled,
  children,
  type = 'button',
  ...props
}: ButtonProps) {
  return (
    <button type={type} className={`btn btn--${variant}`} disabled={disabled || loading} {...props}>
      {loading ? loadingText : children}
    </button>
  );
}
```

src/components/ui/Pagination.tsx

```
import { Button } from './Button';

type PaginationProps = {
  page: number;
  totalPages: number;
  onPageChange: (page: number) => void;
};

/** G09: sólo emite la página pedida; no hace HTTP. Sirve igual para paginación cliente o servidor.
  ↪ */
export function Pagination({ page, totalPages, onPageChange }: PaginationProps) {
  return (
    <nav className="pagination" aria-label="Paginación">
      <Button variant="ghost" disabled={page <= 1} onClick={() => onPageChange(page - 1)}>
        Anterior
      </Button>
      <span>
        Página {page} de {totalPages}
      </span>
      <Button variant="ghost" disabled={page >= totalPages} onClick={() => onPageChange(page + 1)}>
        Siguiente
      </Button>
    </nav>
  );
}
```

src/components/ui/Modal.tsx

```
import { useEffect, useRef, type ReactNode } from 'react';

type ModalProps = {
  open: boolean;
  title: string;
  children: ReactNode;
  onClose: () => void;
};

/**
 * G09: modal con el <dialog> nativo. showModal() lo coloca encima de todo, bloquea el fondo
 * y permite cerrarlo con Escape (evento cancel). El Effect sincroniza la prop open con el DOM.
 */
export function Modal({ open, title, children, onClose }: ModalProps) {
  const ref = useRef<HTMLDialogElement>(null);

  useEffect(() => {
    const dialog = ref.current;
    if (!dialog) return;
    if (open && !dialog.open) dialog.showModal();
    if (!open && dialog.open) dialog.close();
  }, [open]);

  return (
    <dialog
      ref={ref}
      className="modal"
      aria-labelledby="modal-title"
      onCancel={(e) => {
        e.preventDefault();
        onClose();
      }}
    >
      <header className="modal__header">
        <h2 id="modal-title">{title}</h2>
        <button type="button" className="modal__close" aria-label="Cerrar" onClick={onClose}>
          x
        </button>
      </header>
      {children}
    </dialog>
  );
}
```

src/components/ui/ConfirmDialog.tsx

```
import { Button } from './Button';
import { Modal } from './Modal';

type ConfirmDialogProps = {
  open: boolean;
  title: string;
  message: string;
  confirmLabel?: string;
  confirming?: boolean;
  onConfirm: () => void;
  onCancel: () => void;
};

/** G09: reemplaza window.confirm(). No sabe QUÉ se elimina: la Page decide qué service llamar. */
export function ConfirmDialog({
  open,
  title,
  message,
  confirmLabel = 'Eliminar',
  confirming = false,
  onConfirm,
  onCancel,
}: ConfirmDialogProps) {
  return (
    <Modal open={open} title={title} onClose={onCancel}>
      <p>{message}</p>
      <div className="modal__actions">
        <Button variant="secondary" onClick={onCancel} disabled={confirming}>
          Cancelar
        </Button>
        <Button variant="danger" loading={confirming} loadingText="Eliminando..."
          ↪ onClick={onConfirm}>
          {confirmLabel}
        </Button>
      </div>
    </Modal>
  );
}
```

También están `SearchInput`, `SelectFilter`, `StatusBadge`, `EmptyState`, `MetricCard` y `Alert`, con la misma idea.

✓ Qué deberías ver

Al pulsar **Eliminar** aparece un cuadro de confirmación sobre la página oscurecida; **Escape** lo cierra.

⚠ Si falla

En las pruebas, `dialog.showModal` is not a function: jsdom no implementa `<dialog>`. Se simula en `src/test/setup.ts` (paso 17), no en el componente.

i Para entenderlo

`ConfirmDialog` reemplaza a `window.confirm()`: se puede estilizar, probar y mostrar el estado *Eliminando...* mientras espera al backend. `Button` tiene `type="button"` por defecto para que un botón dentro de un formulario no lo envíe sin querer.

Paso 11 · La tabla (G03)

La tabla recibe la lista ya filtrada y paginada, y emite intenciones (`onEdit`, `onDelete`):

`src/features/categorias/components/CategoriaTable.tsx` (fragmento)

```
interface CategoriaTableProps {
  categorias: Categoria[];
  /** Cantidad de productos por categoría (dato derivado que calcula la Page). */
  productosPorCategoria?: Map<number, number>;
  onEdit: (id: number) => void;
  onDelete: (categoria: Categoria) => void;
  deletingId?: number | null;
}

// ...


  {categorias.map((categoria) => (
    <tr key={categoria.id}>
      <td>{categoria.id}</td>
      <td className="code-cell">{categoria.codigo}</td>
      <td>{categoria.nombre}</td>
      <td className="muted-cell">{categoria.descripcion ?? '-'}</td>
      <td>{productosPorCategoria?.get(categoria.id) ?? 0}</td>
      <td>
        <StatusBadge active={categoria.activo} />
      </td>
      <td className="actions-cell">
        <Button variant="secondary" onClick={() => onEdit(categoria.id)}
          ↵ aria-label={`Editar ${categoria.nombre}`}>
          Editar
        </Button>
        <Button
          variant="danger"
          onClick={() => onDelete(categoria)}
          loading={deletingId === categoria.id}
          loadingText="Eliminando..."
          aria-label={`Eliminar ${categoria.nombre}`}
        >
          Eliminar
        </Button>
      </td>
    </tr>
  ))}
</tbody>
```

✓ Qué deberías ver

Una fila por categoría, con la cantidad de productos de cada una.

⚠ Si falla

Advertencia *Each child in a list should have a unique "key" prop*: falta `key` en el `<tr>`. Usa el `id`, nunca el índice del arreglo.

i Para entenderlo

`key` le dice a React qué fila es cuál cuando la lista cambia (al borrar o filtrar). `aria-label="Editar Material de oficina"` hace que cada botón sea distinguible para un lector de pantalla y para las pruebas. `ProductoTable.tsx` es igual, y además formatea el precio con `Intl.NumberFormat('es-B0')`.

Paso 12 · Formulario controlado y validación (G04)

Primero, las reglas. Son las mismas del `CrearCategoriaRequest` del backend:

```
src/features/categorias/utils/categoriaValidation.ts
```

```
import type { CategoriaFormData } from '../types/CategoriaFormData';

export type CategoriaFormErrors = Partial<Record<keyof CategoriaFormData, string>>;

const CODIGO = /^[A-Za-z0-9-]+$/;

/**
 * G04: mismas reglas que CrearCategoriaRequest del backend (@NotBlank, @Size, @Pattern).
 * Mejoran la experiencia (error inmediato junto al campo), pero NO reemplazan la validación
 * de Spring Boot ni las constraints de PostgreSQL.
 */
export function validarCategoria(data: CategoriaFormData): CategoriaFormErrors {
  const errors: CategoriaFormErrors = {};
  const codigo = data.codigo.trim();
  if (!codigo) errors.codigo = 'El código es obligatorio';
  else if (codigo.length > 30) errors.codigo = 'El código admite máximo 30 caracteres';
  else if (!CODIGO.test(codigo)) errors.codigo = 'Sólo letras, números y guiones';

  const nombre = data.nombre.trim();
  if (!nombre) errors.nombre = 'El nombre es obligatorio';
  else if (nombre.length > 100) errors.nombre = 'El nombre admite máximo 100 caracteres';

  if (data.descripcion.length > 1000) errors.descripcion = 'La descripción admite máximo 1000
    ↵ caracteres';
  return errors;
}
```

Después, el formulario. Un solo componente sirve para crear y para editar:

src/features/categorias/components/CategoriaForm.tsx (fragmento)

```

/** G06: copia la categoría a editar al estado editable (o el formulario vacío en modo crear). */
function aFormulario(categoria: Categoria | null): CategoriaFormData {
  return categoria
    ? { codigo: categoria.codigo, nombre: categoria.nombre, descripcion: categoria.descripcion ??
      ↪ '', activo: categoria.activo }
    : formularioVacio;
}

// ...

export default function CategoriaForm({ categoria, onSave, onCancelEdit }: CategoriaFormProps) {
  const [formData, setFormData] = useState<CategoriaFormData>(() => aFormulario(categoria));
  const [errors, setErrors] = useState<CategoriaFormErrors>({});
  const [saving, setSaving] = useState(false);
  const [apiError, setApiError] = useState('');
  const isEditing = categoria !== null;

  // ...

  const handleChange = (e: ChangeEvent<HTMLInputElement | HTMLTextAreaElement>) => {
    const { name, value } = e.target;
    const valor = e.target instanceof HTMLInputElement && e.target.type === 'checkbox' ?
      ↪ e.target.checked : value;
    // Inmutabilidad: se crea un objeto nuevo con ...prev; nunca formData.nombre = ...
    setFormData((prev) => ({ ...prev, [name]: valor }));
  };

  // ...

  const handleSubmit = async (e: FormEvent<HTMLFormElement>) => {
    e.preventDefault(); // la SPA no recarga la página
    setApiError('');
    const validationErrors = validarCategoria(formData);
    setErrors(validationErrors);
    if (Object.keys(validationErrors).length > 0) return; // no se envía una petición que fallaría

    const base = {
      codigo: formData.codigo.trim(),
      nombre: formData.nombre.trim(),
      descripcion: formData.descripcion.trim() || null,
    };
    try {
      setSaving(true);
      if (categoria) {
        onSave(await categoriaService.actualizar(categoria.id, { ...base, activo: formData.activo
          ↪ }), 'edit');
      } else {
        onSave(await categoriaService.crear(base), 'create');
      }
    }
  };

```

src/features/categorias/components/CategoriaForm.tsx (fragmento) (continuación)

```

    setFormData(formularioVacio);
  }
} catch (err) {
  // 400 del backend: se muestran sus errores junto a cada campo; 409: código o nombre repetido
  if (err instanceof ApiError) setErrors(err.fieldErrors as CategoriaFormErrors);
  setApiError(mensajeDeError(err, 'No se pudo guardar la categoría'));
} finally {
  setSaving(false);
}
};

```

✓ Qué deberías ver

Al pulsar **Crear categoría** con todo vacío aparece *El código es obligatorio* debajo del campo, y en **Network** no sale ninguna petición.

⚠ Si falla

Si al tipear no cambia el texto del input, falta `onChange` o el `name` del input no coincide con la propiedad de `formData`.

i Para entenderlo

Input controlado: el valor del input sale del estado (`value={formData.codigo}`) y cada tecla actualiza el estado. React es la única fuente de verdad.

La validación del frontend sólo **mejora la experiencia**: el backend vuelve a validar (`@Valid`) y PostgreSQL tiene sus constraints. Si algo pasa igual, el 400 del backend trae `fieldErrors` y se muestran en el mismo lugar.

Diferencia con G04/G06: las guías copian la categoría al estado con un `useEffect`. Aquí el estado inicial se calcula con `aFormulario(categoria)` y la Page pone `key={editing?.id}`: cuando cambia lo que se edita, React crea un formulario nuevo. Es lo que recomienda React, y oxcint marca el patrón de las guías (`set-state-in-effect`).

Paso 13 · La Page: CRUD completo de Categoría (G06)

La Page coordina todo: carga, edición, eliminación y mensajes. Las piezas de los pasos anteriores se juntan aquí:

src/features/categorias/pages/CategoriasPage.tsx (fragmento)

```
const { data: categorias, setData: setCategorias, loading, error, setError } = useCategorias();
const { data: productos } = useProductos();

const [editing, setEditing] = useState<Categoria | null>(null);
const [loadingDetail, setLoadingDetail] = useState(false);
const [pendingDelete, setPendingDelete] = useState<Categoria | null>(null);
const [deletingId, setDeletingId] = useState<number | null>(null);
const [success, setSuccess] = useState('');

// ...

// G06: GET /{id} antes de editar, para trabajar con la versión actual del servidor
const handleEdit = async (id: number) => {
  try {
    setLoadingDetail(true);
    setError('');
    setEditing(await categoriaService.obtenerPorId(id));
  } catch (err) {
    setError(mensajeDeError(err, 'No se pudo cargar la categoría'));
  } finally {
    setLoadingDetail(false);
  }
};

// ...

// Sincronización local con la RESPUESTA del backend (trae el id real y los datos normalizados)
const handleSaved = (saved: Categoria, mode: 'create' | 'edit') => {
  setCategorias((prev) => (mode === 'create' ? [...prev, saved] : prev.map((c) => (c.id ===
    ↪ saved.id ? saved : c))));
  setEditing(null);
  mostrarExito(mode === 'create' ? `Categoría ${saved.codigo} creada` : `Categoría
    ↪ ${saved.codigo} actualizada`);
};

// ...

const confirmDelete = async () => {
  if (!pendingDelete) return;
  const categoria = pendingDelete;
  try {
    setDeletingId(categoria.id);
    setError('');
    await categoriaService.eliminar(categoria.id);
    // Sólo se quita la fila DESPUÉS de que el backend confirmó (204)
    setCategorias((prev) => prev.filter((c) => c.id !== categoria.id));
    if (editing?.id === categoria.id) setEditing(null);
    mostrarExito(`Categoría ${categoria.codigo} eliminada`);
  }
};
```

src/features/categorias/pages/CategoriasPage.tsx (fragmento) (continuación)

```

    } catch (err) {
      // 409: tiene productos. La fila NO se borra de la pantalla: el backend la rechazó.
      setError(mensajeDeError(err, 'No se pudo eliminar la categoría'));
    } finally {
      setDeletingId(null);
      setPendingDelete(null);
    }
  };

  // ...

  <CategoriaForm key={editing?.id ?? 'nueva'} categoria={editing} onSave={handleSaved}
    ↪ onCancelEdit={() => setEditing(null)} />

```

INVENTARIO, COMPRAS INTERNAS Y MOVIMIENTOS DE ALMACÉN

StockFlow React + TypeScript

ENTIDAD PADRE · RF-06

Categorías

CRUD completo conectado con Spring Boot (/api/categorias).

La categoría con id 1 tiene productos asociados y no se puede eliminar; desactívala con PUT (activo = false)

Nueva categoría

Código: CAT-OFI Nombre: Material de oficina

Descripción:

Crear categoría

Buscar categoría: Código, nombre o descripción Estado: Todas Tamaño: 10 por página

ID	CÓDIGO	NOMBRE	DESCRIPCIÓN	PRODUCTOS	ESTADO	ACCIONES
3	CAT-EPP	Seguridad industrial	Equipos de protección personal	1	Activo	Editar Eliminar
2	CAT-LIM	Limpieza	Insumos de limpieza e higiene	1	Activo	Editar Eliminar
1	CAT-OFI	Material de	Panelería v útiles	2	Activo	Editar Eliminar

Intento de eliminar una categoría con productos: el backend responde 409 y la fila **sigue** en la tabla.

✓ Qué deberías ver

Crear, editar y eliminar funcionan, y cada acción muestra un mensaje verde. Al eliminar **CAT-OFI** aparece el 409 del backend en rojo.

⚠ Si falla

Si después de crear la fila no aparece hasta recargar, falta actualizar el estado con la respuesta (**handleSaved**).

i Para entenderlo

Editar hace primero `GET /{id}`: así se edita la versión actual del servidor, no la que quedó en la tabla.

Después de guardar, se usa la respuesta del backend (trae el `id` real y los datos normalizados) en lugar de volver a pedir la lista entera.

La fila se quita sólo después del 204. Si el backend responde 409 (la categoría tiene productos), la fila se queda y se muestra su mensaje: la pantalla nunca contradice a la base de datos.

Paso 14 · La entidad hija: Producto y la relación 1:N (G07)

Producto repite el patrón de Categoría y agrega la relación. En el formulario, el usuario elige la categoría por su nombre, pero se envía su `id`:

src/features/productos/components/ProductoForm.tsx (fragmento)

```
/** Convierte el texto del formulario en el payload tipado. Sólo se llama DESPUÉS de validar. */
function aPayload(data: ProductoFormData): ProductoCreateRequest {
  return {
    categoriaId: Number(data.categoriaId),
    codigo: data.codigo.trim(),
    nombre: data.nombre.trim(),
    descripcion: data.descripcion.trim() || null,
    unidadMedida: data.unidadMedida,
    stockMinimoDefault: Number(data.stockMinimoDefault),
    precioReferencial: data.precioReferencial.trim() === '' ? null : Number(data.precioReferencial),
  };
}

// ...

// Opciones del select: categorías activas + la actual del producto (aunque esté inactiva, para
// ↪ no perderla)
const opciones = categorias.filter((c) => c.activo || c.id === producto?.categoriaId);

// ...

<select name="categoriaId" value={formData.categoriaId} onChange={handleChange}>
  <option value="">Seleccione una categoría</option>
  {opciones.map((c) => (
    <option key={c.id} value={String(c.id)}>
      {c.codigo} · {c.nombre}
      {c.activo ? '' : ' (inactiva)'}
    </option>
  ))}
</select>
```

La validación agrega la regla de la relación (la misma que aplica `ProductoService` en el backend):

src/features/productos/utils/productoValidation.ts (fragmento)

```
const categoriaId = Number(data.categoriaId);
const categoria = categorias.find((c) => c.id === categoriaId);
if (!data.categoriaId) errors.categoriaId = 'Seleccione una categoría';
else if (!categoria) errors.categoriaId = 'La categoría seleccionada no existe';
else if (!categoria.activo && categoria.id !== categoriaActualId) {
  errors.categoriaId = 'La categoría está inactiva y no admite productos nuevos';
}
```

Y en la Page, un `Map` resuelve el nombre de la categoría de cada fila sin hacer una petición por fila:

src/features/productos/pages/ProductosPage.tsx (fragmento)

```
// G07: Map id -> Categoria para resolver nombres sin una petición por fila
const categoriasPorId = useMemo(
  () => new Map<number, Categoria>(categorias.data.map((c) => [c.id, c])),
  [categorias.data],
);

// ...

const filtrados = productos.data.filter((p) => {
  const termino = normalizar(search);
  const categoria = categoriasPorId.get(p.categoriaId);
  const coincideTexto =
    !termino || [p.codigo, p.nombre, categoria?.nombre, categoria?.codigo].some((c) =>
      ↪ normalizar(c).includes(termino));
  // El select entrega string: se convierte a number para comparar con categoriaId
  const coincideCategoria = categoriaFiltro === 'all' || p.categoriaId ===
    ↪ Number(categoriaFiltro);
  const coincideEstado = status === 'all' || (status === 'active' ? p.activo : !p.activo);
  return coincideTexto && coincideCategoria && coincideEstado;
});
const pagina = paginar(filtrados, page, pageSize);
```

✓ Qué deberías ver

El select muestra las categorías activas (CAT-OFI · **Material de oficina**); la tabla, la columna **Categoría** con código y nombre; y el filtro por categoría deja sólo sus productos.

⚠ Si falla

Si el filtro por categoría no encuentra nada, se está comparando un **string** (lo que entrega el select) con un **number** (**categoriaId**): convierte con **Number(...)**.

i Para entenderlo

La relación 1:N **la guarda el backend** (clave foránea **producto.categoria_id**). El frontend sólo envía el número correcto y muestra el error si el backend la rechaza (422 si la categoría está inactiva).

Al editar un producto cuya categoría se desactivó, esa categoría se agrega a las opciones con (*inactiva*): si no, el select quedaría vacío y el usuario la perdería sin querer.

Paso 15 · Búsqueda, filtros y paginación (G09)

La lista que se muestra se **calcula** en cada render a partir de los datos y de lo que eligió el usuario:

src/shared/utils/paginar.ts

```
/** G09: paginación en el cliente. Se aplica DESPUÉS de buscar y filtrar. */
export interface ResultadoPagina<T> {
  items: T[];
  pagina: number;
  totalPaginas: number;
  totalItems: number;
}

export function paginar<T>(datos: T[], pagina: number, tamaño: number): ResultadoPagina<T> {
  const totalPaginas = Math.max(1, Math.ceil(datos.length / tamaño));
  // Si un filtro o un DELETE dejó la página fuera de rango, se ajusta a la última válida
  const paginaValida = Math.min(Math.max(1, pagina), totalPaginas);
  const inicio = (paginaValida - 1) * tamaño;
  return {
    items: datos.slice(inicio, inicio + tamaño),
    pagina: paginaValida,
    totalPaginas,
    totalItems: datos.length,
  };
}

/** Búsqueda sin distinguir mayúsculas ni tildes ("categoria" encuentra "Categoría"). */
export function normalizar(texto: string | null | undefined): string {
  return (texto ?? '').normalize('NFD').replace(/[\u0300-\u036f]/g, '').toLowerCase().trim();
}
```

src/features/categorias/pages/CategoriasPage.tsx (fragmento)

```
const filtradas = categorias.filter((c) => {
  const termino = normalizar(search);
  const coincideTexto =
    !termino || [c.codigo, c.nombre, c.descripcion].some((campo) =>
      ⇐ normalizar(campo).includes(termino));
  const coincideEstado = status === 'all' || (status === 'active' ? c.activo : !c.activo);
  return coincideTexto && coincideEstado; // AND entre filtros, OR entre campos de texto
});
const pagina = paginar(filtradas, page, pageSize); // primero filtrar, después paginar

// ...

<SearchInput
  value={search}
  label="Buscar categoría"
  placeholder="Código, nombre o descripción"
  onChange={(v) => {
    setSearch(v);
    setPage(1); // al cambiar el filtro se vuelve a la primera página
  }}
/>
```

✓ Qué deberías ver

Escribir **categoria** encuentra *Categoría*... aunque no lleve tilde; al cambiar un filtro se vuelve a la página 1.

⚠ Si falla

Si al filtrar quedas en una página vacía (*Página 3 de 1*), se paginó antes de filtrar o no se volvió a la página 1. **paginar** además ajusta una página fuera de rango.

i Para entenderlo

Datos derivados: la lista filtrada **no** se guarda en otro `useState`, se calcula. Así nunca queda desincronizada con los datos.

Diferencia con G09: la guía vuelve a la página 1 con un `useEffect` que observa los filtros. Aquí se hace en el mismo evento que cambia el filtro (`setPage(1)` en el `onChange`): menos renders y el código dice qué pasa y cuándo.

Paso 16 · Dashboard y errores globales (G10)

El panel de inicio reutiliza los hooks: las métricas se **cuentan** a partir de las listas que ya existen.

src/pages/DashboardPage.tsx (fragmento)

```
export default function DashboardPage() {
  const categorias = useCategorias();
  const productos = useProductos();
  const stockBajo = useAsyncList(inventarioService.stockBajoMinimo);

  if (categorias.loading || productos.loading || stockBajo.loading) {
    return <div className="state-card">Cargando panel...</div>;
  }
  // || y no ??: un error vacío es '' (no null), así que ?? nunca pasaría al siguiente
  const error = categorias.error || productos.error || stockBajo.error;

  const metricas = {
    categorias: categorias.data.length,
    categoriasActivas: categorias.data.filter((c) => c.activo).length,
    productos: productos.data.length,
    productosActivos: productos.data.filter((p) => p.activo).length,
    alertas: stockBajo.data.length,
  };
};
```

The screenshot shows the StockFlow dashboard. On the left is a dark blue sidebar with the 'StockFlow Inventario' logo and navigation links for 'Inicio', 'Categorías', and 'Productos'. The main content area has a light blue header with the title 'StockFlow' and a 'React + TypeScript' badge. Below the header, it says 'INVENTARIO, COMPRAS INTERNAS Y MOVIMIENTOS DE ALMACÉN'. The main section is titled 'Resumen del inventario' and includes the text 'Datos en vivo desde el backend Spring Boot.' There are three cards: 'Categorías' showing 5 (5 activas), 'Productos' showing 5 (5 activos), and 'Alertas de stock mínimo' showing 1 (1 existencias en o bajo su mínimo). Below these is a section 'Stock bajo (RF-20)' with the text 'Existencias por producto y ubicación que alcanzaron su stock mínimo.' and a table:

PRODUCTO	UBICACIÓN	CANTIDAD	MÍNIMO
PRD-OFI-002 · Bolígrafo azul	Depósito Norte	25	100

At the bottom, it says 'Gestiona el catálogo en [Categorías](#) y [Productos](#).'

El dashboard con datos reales del backend: 5 categorías, 5 productos y una alerta de stock mínimo.

Y un *error boundary* evita que un error de programación deje la pantalla en blanco:

src/app/AppErrorBoundary.tsx

```
import { Component, type ErrorInfo, type ReactNode } from 'react';

type Props = { children: ReactNode };
type State = { hasError: boolean };

/**
 * G10: captura errores que ocurren al RENDERIZAR (por ejemplo, leer una propiedad de undefined).
 * NO captura errores de fetch ni de event handlers: esos se manejan con try/catch y estado.
 */
export class AppErrorBoundary extends Component<Props, State> {
  state: State = { hasError: false };

  static getDerivedStateFromError(): State {
    return { hasError: true };
  }

  componentDidCatch(error: unknown, info: ErrorInfo) {
    console.error('Error de render', error, info.componentStack);
  }

  render() {
    if (this.state.hasError) {
      return (
        <main className="fatal-error" role="alert">
          <h1>La interfaz tuvo un problema.</h1>
          <p>Recarga la página. Si el error persiste, revisa la consola del navegador (F12).</p>
        </main>
      );
    }
    return this.props.children;
  }
}
```

✓ Qué deberías ver

Las tres tarjetas con números y la tabla **Stock bajo (RF-20)** con *Bolígrafo azul* en *Depósito Norte* (25 de 100).

i Para entenderlo

Diferencia con G10: la guía combina los errores con `clientes.error ?? vehiculos.error`. Pero un error vacío es `''`, que **no** es *nullish*: `??` siempre devuelve el primero, aunque esté vacío, y el segundo error nunca se muestra. Con `||` sí funciona.

El error boundary captura errores **de render**. Los errores de `fetch` no llegan ahí: esos se manejan con `try/catch` y el estado `error` de cada pantalla.

Paso 17 · Pruebas automáticas con Vitest (G10)

La configuración de pruebas va dentro de `vite.config.ts`, junto con el puerto fijo:

`vite.config.ts`

```
import react from '@vitejs/plugin-react'
import { defineConfig } from 'vite'

// https://vite.dev/config/ · https://vitejs.dev/config/
export default defineConfig({
  plugins: [react()],
  server: {
    // El CorsConfig del backend sólo permite http://localhost:5173. Si el puerto está ocupado,
    // es mejor que Vite se detenga con un error claro a que use 5174 y todo falle por CORS.
    port: 5173,
    strictPort: true,
  },
  test: {
    // G10: jsdom simula el navegador (document, window) para renderizar componentes
    environment: 'jsdom',
    setupFiles: ['./src/test/setup.ts'],
    // Vitest corre en modo "test" y no lee .env.development: se define aquí la URL de la API
    env: { VITE_API_URL: 'http://localhost:8080/api' },
  },
})
```

`src/test/setup.ts`

```
// G10: agrega matchers como toBeInTheDocument(), toBeDisabled() y toHaveClass() a expect
import '@testing-library/jest-dom/vitest';
import { cleanup } from '@testing-library/react';
import { afterEach } from 'vitest';

// Sin "globals: true", Testing Library no desmonta solo: se limpia el DOM después de cada prueba
afterEach(() => {
  cleanup();
});

// jsdom no implementa showModal()/close() de <dialog>; el navegador real sí.
// Se simulan con el atributo open para poder probar Modal y ConfirmDialog.
HTMLDialogElement.prototype.showModal ??= function (this: HTMLDialogElement) {
  this.setAttribute('open', '');
};
HTMLDialogElement.prototype.close ??= function (this: HTMLDialogElement) {
  this.removeAttribute('open');
};
```

Una prueba de Page reemplaza los services por versiones falsas (*mocks*), así no depende del backend:

src/features/categorias/pages/CategoriasPage.test.tsx (fragmento)

```
// G10: se reemplazan los services; la prueba no depende de que el backend esté encendido
vi.mock('../services/categoriaService');
vi.mock('../products/services/productoService');

beforeEach(() => {
  vi.resetAllMocks();
  vi.mocked(categoriaService.listar).mockResolvedValue(categoriasMock);
  vi.mocked(productoService.listar).mockResolvedValue(productosMock);
});

// ...

it('conserva la fila y muestra el mensaje del backend cuando DELETE responde 409', async () => {
  vi.mocked(categoriaService.eliminar).mockRejectedValue(
    new ApiError(409, 'La categoría tiene productos asociados y no puede eliminarse'),
  );
  render(<CategoriasPage />);
  await userEvent.click(await screen.findByRole('button', { name: 'Eliminar Material de oficina'
    ↵ }));
  const dialogo = screen.getByRole('dialog');
  await userEvent.click(within(dialogo).getByRole('button', { name: 'Eliminar' }));

  expect(await screen.findByText('La categoría tiene productos asociados y no puede
    ↵ eliminarse')).toBeInTheDocument();
  expect(screen.getByText('CAT-OFI')).toBeInTheDocument();
});
```

Terminal

```
npm run test:run
```

✓ Qué deberías ver

Resultado real:

- ✓ **Test Files 8 passed (8)**
- ✓ **Tests 28 passed (28)**: componentes de UI, validaciones, paginación, `apiClient` y la página de categorías (listado, búsqueda, DELETE con 409 y con 204).

⚠ Si falla

Found multiple elements with the role "button": el DOM de una prueba anterior sigue ahí. Falta el `cleanup()` del `setup.ts`.

i Para entenderlo

Diferencias con G10: la guía usa un `vitest.config.ts` aparte. Se unió a `vite.config.ts` porque, si existe `vitest.config.ts`, Vitest ignora `vite.config.ts` y habría que repetir el plugin de React.

La guía tampoco activa `globals`; sin eso, Testing Library **no** limpia el DOM entre pruebas, y las pruebas se pisan. Por eso `setup.ts` llama a `cleanup()`.

`test.env` define `VITE_API_URL`, porque Vitest corre en modo `test` y no lee `.env.development`.

Paso 18 · Verificación final contra el backend real

Con el backend encendido, los cuatro comandos que deben pasar antes de entregar:

```
npm run lint
npm run test:run
npm run build
npm run dev
```

Además se recorrió la aplicación con Chrome automatizado contra el backend real. Estos 18 controles pasaron:

CÓDIGO	NOMBRE	CATEGORÍA	UNIDAD	STOCK MÍN.	PRECIO REF.	ESTADO	ACCIONES
PRD-EPP-001	Guantes de nitrilo	CAT-EPP · Seguridad industrial	CAJA	15	95,00	Activo	Editar Eliminar
PRD-LIM-001	Detergente líquido 5L	CAT-LIM · Limpieza	LITRO	10	75,00	Activo	Editar Eliminar
PRD-OFI-001	Resma papel bond A4 75g	CAT-OFI · Material de oficina	PAQUETE	20	38,50	Activo	Editar Eliminar
PRD-OFI-002	Bolígrafo azul	CAT-OFI · Material de oficina	UNIDAD	100	2,50	Activo	Editar Eliminar
PRD-TEC-001	Tóner negro HP 85A	CAT-TEC · Tecnología	UNIDAD	5	420,00	Activo	Editar Eliminar

La tabla de productos, con la categoría resuelta por el `Map` y el precio formateado.

✓ Qué deberías ver

Controles verificados el 5 de octubre de 2026:

- ✓ Dashboard con métricas reales; listado de categorías (`GET /api/categorias`) y de productos (`GET /api/productos`).
- ✓ Validación local sin petición (código obligatorio; RN-07: stock mínimo negativo).
- ✓ `POST` crea y la fila aparece; código repetido → *Ya existe una categoría con el valor: CAT-E2E (409)*.
- ✓ `GET /{id}` + `PUT` actualizan la fila; el formulario precarga el precio 12.50.
- ✓ `DELETE` de una categoría con productos → 409 y la fila se conserva; `DELETE` sin dependencias → 204 y la fila desaparece.
- ✓ Búsqueda sin tildes, filtro por categoría, ruta inexistente → página 404, y diseño adaptado a móvil.
- ✓ Colección de Postman: 139 de 139 pruebas, incluido el nuevo `GET /api/productos`.

⚠ Si falla

Si `npm run lint` muestra advertencias o `npm run build` muestra `error TS...`, corrégelas antes del commit: el build de producción no se genera con errores de tipos.

4. Diferencias con las guías y por qué

Las guías se escribieron para versiones algo anteriores y tienen algunas inconsistencias entre sí. Esta tabla resume cada desvío, para que puedas justificarlo si te preguntan por qué el código no es idéntico a la guía.

Guía	Lo que dice la guía	El problema	Qué se hizo en StockFlow
G01	<code>VITE_API_BASE_URL=http://localhost:8080</code>	G05 y G10 usan <code>VITE_API_URL</code> y con <code>/api</code> al final.	Un solo nombre: <code>VITE_API_URL=http://localhost:8080/api</code> .
G01	ESLint (<code>eslint.config.*</code>)	La plantilla de Vite 8 trae oxlint .	<code>npm run lint</code> ejecuta oxlint, con la regla de hooks de React activada.
G01-G05	<code>services/http</code> , <code>api/</code> , <code>models/</code> , <code>types/</code> según la guía	Las guías no coinciden en las carpetas.	<code>api/apiClient.ts</code> único; <code>models/</code> = lo que responde el backend; <code>types/</code> = requests y formularios.
G05	<code>constructor(public status: number, ...)</code> en <code>ApiError</code>	Error TS1294 con <code>erasableSyntaxOnly</code> (activo en la plantilla).	Campos <code>readonly</code> declarados y asignados en el constructor.
G04, G06	<code>useEffect</code> que copia la entidad a editar al estado del formulario	Render extra; oxlint avisa <code>set-state-in-effect</code> .	Estado inicial con <code>aFormulario()</code> y <code>key={editing?.id}</code> en la Page.
G07	Listar todos los vehículos con <code>GET /api/vehiculos</code>	El backend de StockFlow no tenía un listado general de productos.	Se agregó <code>GET /api/productos</code> en el backend, con pruebas (commit <code>46150a5</code>).
G09	<code>useEffect</code> que vuelve a la página 1 cuando cambia un filtro	Effect innecesario: el cambio nace de un evento.	<code>setPage(1)</code> dentro del <code>onChange</code> de cada filtro.
G10	<code>clientes.error ?? vehiculos.error</code>	<code>''</code> no es <code>nullish</code> : el segundo error nunca se muestra.	<code>categorias.error productos.error stockBajo.error</code> .
G10	<code>vitest.config.ts</code> separado, sin <code>globals</code>	Sin <code>globals</code> , el DOM no se limpia entre pruebas; jsdom no tiene <code>showModal()</code> .	Config en <code>vite.config.ts</code> ; <code>cleanup()</code> y simulación de <code><dialog></code> en <code>setup.ts</code> .
—	Ninguna guía fija el puerto	Si 5173 está ocupado, Vite usa 5174 y todo falla por CORS.	<code>server.port: 5173</code> con <code>strictPort: true</code> (commit <code>a076d23</code>).

5. Cómo explicarlo en la defensa

El recorrido de un clic

Si te piden explicar "qué pasa cuando el usuario pulsa **Crear producto**", este es el camino completo, de la pantalla a la base y de vuelta:

1. `ProductoForm.handleSubmit` evita la recarga (`e.preventDefault()`) y valida con `validarProducto`. Si hay errores, los muestra y **no** envía nada.
2. `aPayload` convierte el texto del formulario en un `ProductoCreateRequest` (números de verdad, `null` en lo opcional).
3. `productoService.crear` llama a `apiFetch('/productos', { method: 'POST', body })`.
4. `apiFetch` arma la URL con `VITE_API_URL`, agrega `Content-Type: application/json` y ejecuta `fetch`.
5. El navegador envía la petición a `localhost:8080`. Como el origen es otro puerto, aplica CORS: `CorsConfig` lo permite.
6. En el backend: `ProductoController` → `@Valid` → `CrearProductoUseCase` → `ProductoService` (reglas: categoría activa, código único) → `ProductoPersistenceAdapter` → Hibernate → `INSERT` en PostgreSQL.
7. El backend responde **201** con el `ProductoResponse` (incluye el `id` generado por PostgreSQL).
8. `apiFetch` ve `response.ok` y devuelve el JSON tipado como `Producto`.
9. `ProductosPage.handleSaved` agrega el producto al estado con `setData(prev => [...prev, saved])`; React vuelve a renderizar la tabla y muestra *Producto ... creado*.
10. Si en el paso 6 algo falla (409 por código repetido, 422 por categoría inactiva), `apiFetch` lanza `ApiError` con el mensaje del backend y el formulario lo muestra en rojo.

Preguntas probables

Pregunta	Respuesta corta
¿Por qué un service y no <code>fetch</code> en el componente?	Para tener las rutas en un solo lugar y que los componentes sólo expresen intenciones. Si cambia una URL, se cambia en un archivo.
¿Por qué <code>fetch</code> necesita <code>response.ok</code> ?	Porque <code>fetch</code> sólo falla si no hay conexión. Un 404 o un 500 también es una respuesta "exitosa" para <code>fetch</code> .
¿Qué hace el cleanup del <code>useEffect</code> ?	Aborta la petición si el componente se desmonta, para no actualizar el estado de una pantalla que ya no existe.
¿Por qué se ven dos peticiones en desarrollo?	Por <code>StrictMode</code> : monta, desmonta y vuelve a montar para detectar Effects sin cleanup. En producción hay una.
¿Por qué la fila no se borra antes de que responda el backend?	Porque el backend puede rechazar el borrado (409). La pantalla nunca debe mostrar algo que la base no hizo.
¿Dónde se guarda la relación Categoría-Producto?	En PostgreSQL, con la clave foránea <code>producto.categoria_id</code> . El frontend sólo envía <code>categoriaId</code> .
¿La validación del frontend es suficiente?	No. Es para la experiencia de uso; el backend valida con <code>@Valid</code> y reglas de negocio, y la base con constraints.
¿Por qué <code>key</code> en las listas?	Para que React identifique cada fila entre renders. Se usa el <code>id</code> , no el índice.
¿Qué es un input controlado?	Uno cuyo valor sale del estado de React (<code>value</code>) y se actualiza con <code>onChange</code> .

Pregunta	Respuesta corta
¿Por qué <code>VITE_API_URL</code> no puede tener secretos?	Porque Vite la copia dentro del JavaScript que descarga el navegador: cualquiera puede verla.
¿Qué es CORS y quién lo aplica?	Una regla del navegador . El backend dice qué orígenes acepta (<code>CorsConfig</code>); Postman y <code>curl</code> no la aplican.
¿Qué prueban las pruebas del frontend?	Componentes, validaciones, paginación, <code>apiClient</code> y la página de categorías con services simulados (<i>mocks</i>).

6. Problemas frecuentes al implementar

Mensajes reales de TypeScript, Vite, Vitest y del navegador, con su causa y solución.

Lo que ves	Qué significa	Cómo solucionarlo
TS1484: 'Categoria' is a type and must be imported using a type-only import when 'verbatimModuleSyntax' is enabled	Importaste un tipo como si fuera un valor.	<code>import type { Categoria } from ...</code> , o <code>import { x, type Y }.</code>
TS1294: This syntax is not allowed when 'erasableSyntaxOnly' is enabled	Usaste <i>parameter properties</i> (<code>constructor(public x)</code>) o un <code>enum</code> .	Declara los campos en la clase; en vez de <code>enum</code> , un arreglo <code>as const</code> (como <code>UNIDADES_MEDIDA</code>).
TS6133: 'x' is declared but its value is never read	Una variable o import que no se usa (<code>noUnusedLocals</code>).	Bórralo. El build no se genera con este error.
TS2307: Cannot find module 'react-router-dom'	Ejemplo antiguo: el paquete ahora se llama <code>react-router</code> .	<code>import { ... } from 'react-router'.</code>
Página en blanco y <i>Falta VITE_API_URL</i> en la consola	No existe <code>.env.development</code> o le falta la variable.	Créalo desde <code>.env.example</code> y reinicia <code>npm run dev</code> .
<i>No se pudo conectar con el backend...</i> (o <i>Failed to fetch</i>)	El backend está apagado o la URL es incorrecta.	Enciende el backend; revisa <code>VITE_API_URL</code> .
<i>blocked by CORS policy</i> en la consola	El origen no es <code>http://localhost:5173</code> (otro puerto, <code>127.0.0.1</code> o <code>npm run preview</code>).	Usa <code>npm run dev</code> y <code>http://localhost:5173</code> .
Error: Port 5173 is already in use	Ya hay otro <code>npm run dev</code> corriendo.	Ciérralo (<code>Ctrl+C</code>) o búscalo con <code>ss -ltnp grep 5173</code> .
La lista se pide una y otra vez (Network)	El <code>loader</code> del hook cambia en cada render.	Pasa una función estable: <code>categoriaService.listar</code> .
<i>Each child in a list should have a unique "key" prop</i>	Falta <code>key</code> en el elemento del <code>map</code> .	<code>key={item.id}</code> .
<code>react(set-state-in-effect)</code> en <code>npm run lint</code>	Un <code>useEffect</code> llama a <code>setState</code> para copiar props al estado.	Calcula el estado inicial y usa <code>key</code> (paso 12).
Vitest: <i>Found multiple elements with the role...</i>	El DOM de la prueba anterior no se limpió.	<code>afterEach(() => cleanup())</code> en <code>setup.ts</code> .
Vitest: <code>dialog.showModal</code> is not a function	<code>jsdom</code> no implementa <code><dialog></code> .	Simúlalo en <code>setup.ts</code> (paso 17).
Vitest: <i>Falta VITE_API_URL</i>	Vitest corre en modo <code>test</code> y no lee <code>.env.development</code> .	<code>test.env</code> en <code>vite.config.ts</code> .

7. Glosario

Término	Significado simple
Componente	Una función que devuelve lo que se ve (JSX). Por ejemplo <code>CategoriaTable</code> .
Props	Los datos que un componente recibe de su padre. No se modifican dentro del hijo.
Estado (<code>useState</code>)	Datos que cambian con el tiempo; al cambiar, React vuelve a dibujar el componente.
Hook	Función de React que empieza con <code>use</code> (<code>useState</code> , <code>useEffect</code>) o una propia que los combina (<code>useAsyncList</code>).
Efecto (<code>useEffect</code>)	Código que sincroniza el componente con algo externo, como una petición HTTP. Puede devolver un <code>cleanup</code> .
Cleanup	La función que devuelve un efecto; se ejecuta al desmontar o antes de repetir el efecto.
Input controlado	Un input cuyo valor viene del estado y se actualiza con <code>onChange</code> .
Dato derivado	Un valor que se calcula a partir del estado (lista filtrada, métricas) en vez de guardarse aparte.
SPA	Aplicación de una sola página: el navegador carga un HTML y React cambia el contenido sin recargar.
Layout / Outlet	La estructura fija (menú y encabezado) y el hueco donde se pinta la página de la ruta actual.
Service	Objeto que agrupa las llamadas HTTP de una entidad (<code>categoriaService</code>).
AbortController	Permite cancelar una petición <code>fetch</code> en curso.
StrictMode	Modo de desarrollo de React que repite montajes y efectos para encontrar errores.
Design tokens	Variables CSS con los colores, radios y sombras del sistema visual.
Mock	Una versión falsa de algo (por ejemplo, un service) que se usa en las pruebas.
jsdom	Un navegador simulado en Node.js donde corren las pruebas de componentes.
oxlint	Analizador de código que detecta errores y malas prácticas (reemplaza a ESLint en Vite 8).